SAUDI ELECTRONIC UNIVERSITY

Database System for a Smart Healthcare Clinic Network

Database Systems Project

Submitted By:

Fatimah Al Towailib

Department:

Computer Science

Project Description:

Design and implement a database system for a smart healthcare clinic network operating in multiple cities. The network manages patient registrations, appointment bookings, doctor schedules, electronic prescriptions, billing, multiple clinics/branches, medical inventory, security for sensitive data, and user roles (patients, doctors, admin staff). The system must support reporting, analytics, and enforce security and privacy protocols.

Conceptual design:

1. Scope & Rationale

This database supports a multi-clinic healthcare network, registering patients, scheduling appointments, issuing electronic prescriptions, tracking medicine inventory per clinic, and generating invoices, all while enforcing role-based access to sensitive data.

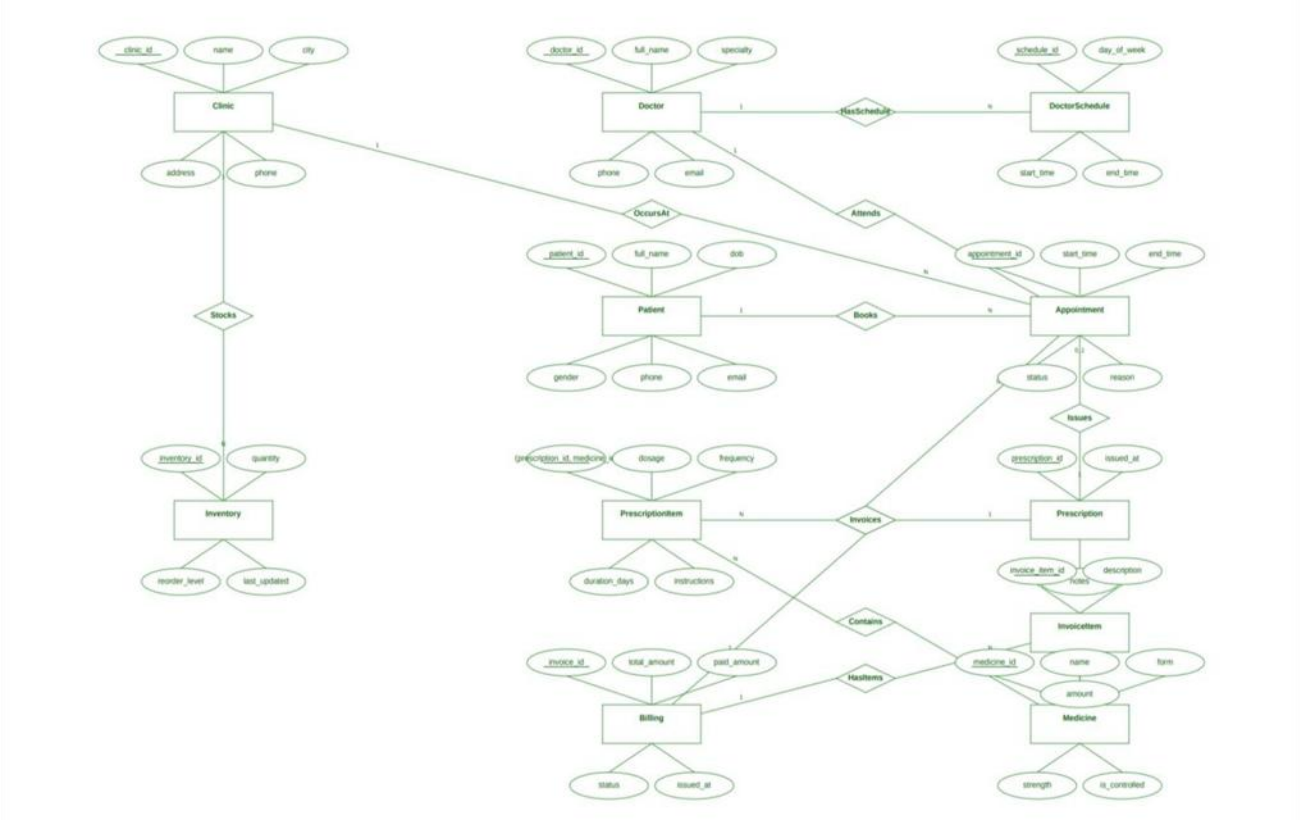
2. Entities & Key Attributes (PK underlined)

- Clinic (clinic_id, name, city, address, phone)
- Doctor (doctor_id, full_name, specialty, phone, email)
- Patient (patient_id, full_name, dob, gender, phone, email)
- DoctorSchedule (schedule_id, day_of_week, start_time, end_time, *FK:* doctor_id, clinic_id)
- Appointment (appointment_id, start_time, end_time, status, reason, *FK:* patient_id, doctor_id, clinic_id)
- Prescription (prescription_id, issued_at, notes, *FK:* appointment_id, doctor_id, patient_id)
- Medicine (medicine_id, name, form, strength, is_controlled)
- PrescriptionItem (prescription_id + medicine_id, dosage, frequency, duration_days, instructions)
- Inventory (inventory_id, quantity, reorder_level, last_updated, *FK:* clinic_id, medicine_id)
- Invoice (Billing) (invoice_id, total_amount, paid_amount, status, issued_at, *FK:* appointment_id, clinic_id, patient_id)
- InvoiceItem (invoice_item_id, description, amount, *FK:* invoice_id)
- User (user_id, username, password_hash, person_type {PATIENT|DOCTOR|STAFF}, person_id, *FK:* role_id)
- Role (role_id, role_name)
- Permission (permission_id, code, description)
- RolePermission (role_id + permission_id)

3. Relationships, Cardinality & Participation:

- Clinic 1–M DoctorSchedule and Doctor 1–M DoctorSchedule (P/P): a clinic/doctor may exist before schedules.
- Patient 1–M Appointment (P/T): appointment must have a patient; a patient may have none yet.
- Doctor 1–M Appointment (P/T): appointment must have a doctor.
- Clinic 1–M Appointment (P/T): appointment happens at one clinic.
- Appointment 1–0..1 Prescription (P/T): some visits have no prescription.
- Prescription M–N Medicine resolved by PrescriptionItem (P/P).
- Clinic M–N Medicine resolved by Inventory (P/P).
- Appointment 1–0..1 Billing (P/T): free/void visits possible.
- Billing 1–M InvoiceItem (T/T): an invoice has one or more items.
- User N–1 Role (T/P); Role M–N Permission via RolePermission (P/P).

4. ER digram :



Pre-Normalization (Complete Tables):

The schema below reflects the initial (unnormalized) design. Redundancy is expected. clinic or doctor names may appear in multiple tables, and some fields may contain repeated or multi-valued data.

Table Name	Attributes (Before Normalization)	Key(s)
Clinic	clinic_id, name, city, address, phone, doctor_names, medicine_names	PK: clinic_id
Doctor	doctor_id, full_name, specialty, phone, email, clinic_name, clinic_city, clinic_phone	PK: doctor_id
Patient	patient_id, full_name, dob, gender, phone, email, last_clinic_name, doctor_name	PK: patient_id
DoctorSchedule	schedule_id, day_of_week, start_time, end_time, doctor_name, clinic_name	PK: schedule_id
Appointment	appointment_id, start_time, end_time, status, reason, patient_name, doctor_name, clinic_name, clinic_address, billed_amount, prescribed_medicines	PK: appointment_id
Prescription	prescription_id, issued_at, notes, appointment_ref, doctor_name, patient_name, medicine_list	PK: prescription_id
Medicine	medicine_id, name, form, strength, is_controlled, common_clinics	PK: medicine_id
PrescriptionItem	prescription_id, medicine_id, medicine_name, dosage, frequency, duration_days, instructions	PK: (prescription_id, medicine_id)

Inventory	inventory_id, clinic_name, medicine_name, quantity, reorder_level, last_updated	PK: inventory_id
Invoice (Billing)	invoice_id, total_amount, paid_amount, status, issued_at, appointment_ref, clinic_name, patient_name	PK: invoice_id
InvoiceItem	invoice_item_id, invoice_ref, description, amount	PK: invoice_item_id
Users	user_id, username, password_hash, person_type, person_id, role_name	PK: user_id
Role	role_id, role_name, description, default_permissions	PK: role_id
Permission	permission_id, code, description	PK: permission_id
RolePermission	role_id, permission_id, permission_code	PK: (role_id, permission_id)

Normalization (Complete Tables, up to 3NF):

After normalization, tables store only attributes that depend on their primary keys, many-to-many relationships are resolved into bridge tables, and foreign keys define clear links between entities.

Table Name	Attributes (After Normalization)	Key(s) and Constraints
Clinic	clinic_id, name, city, address, phone	PK: clinic_id
Doctor	doctor_id, full_name, specialty, phone, email	PK: doctor_id
Patient	patient_id, full_name, dob, gender, phone, email	PK: patient_id

DoctorSchedule	schedule_id, day_of_week, start_time, end_time, doctor_id, clinic_id	PK: schedule_id; FK: doctor_id→Doctor, clinic_id→Clinic
Appointment	appointment_id, patient_id, doctor_id, clinic_id, start_time,	PK: appointment_id; FK: patient_id→Patient,
	end_time, status, reason	doctor_id→Doctor, clinic_id→Clinic
Prescription	prescription_id, appointment_id, doctor_id, patient_id, issued_at, notes	PK: prescription_id; FK: appointment_id→Appointment, doctor_id→Doctor, patient_id→Patient
Medicine	medicine_id, name, form, strength, is_controlled	PK: medicine_id
PrescriptionItem	prescription_id, medicine_id, dosage, frequency, duration_days, instructions	PK: (prescription_id, medicine_id); FK: prescription_id→Prescription, medicine_id→Medicine
Inventory	inventory_id, clinic_id, medicine_id, quantity, reorder_level, last_updated	PK: inventory_id; FK: clinic_id→Clinic, medicine_id→Medicine; UNIQUE(clinic_id, medicine_id)
Invoice	invoice_id, appointment_id, clinic_id, patient_id, total_amount, paid_amount, status, issued_at	PK: invoice_id; FK: appointment_id→Appointment, clinic_id→Clinic, patient_id→Patient
InvoiceItem	invoice_item_id, invoice_id, description, amount	PK: invoice_item_id; FK: invoice_id→Invoice
User	user_id, username, password_hash, person_type, person_id, role_id	PK: user_id; FK: role_id→Role

Role	role_id, role_name	PK: role_id; UNIQUE(role_name)
Permission	permission_id, code, description	PK: permission_id; UNIQUE(code)
RolePermission	role_id, permission_id	PK: (role_id, permission_id); FK: role_id→Role, permission_id→Permission

Normalization Justification

Step	Explanation
1NF	Removed repeating groups and multi-valued attributes (e.g., lists of names or medicines). Each column holds a single value.
2NF	Eliminated partial dependencies by ensuring each non-key attribute depends on the whole key. Separated clinic details from Doctor/Appointment into Clinic.
3NF	Removed transitive dependencies so non-key attributes depend only on the primary key. Resolved M:N relationships using bridge tables like PrescriptionItem and RolePermission.

DBMS Implementation:

- **Creating the tables:**

```
CREATE TABLE Clinic (
    name VARCHAR(100) NOT NULL,
    city VARCHAR(60),
    address VARCHAR(200),
    phone VARCHAR(20) NOT NULL,
    clinic_id INT PRIMARY KEY);
```

```
CREATE TABLE Doctor (
    full_name VARCHAR(100) NOT NULL,
    specialty VARCHAR(80),
    phone VARCHAR(20) NOT NULL,
    email VARCHAR(100),
    doctor_id INT PRIMARY KEY,
    c_ID INT NOT NULL,
    FOREIGN KEY (c_ID) REFERENCES Clinic(clinic_id));

    CREATE TABLE Patient (
    full_name VARCHAR(100) NOT NULL,
    dob DATE NOT NULL,
    gender VARCHAR(10),
    phone VARCHAR(20),
    email VARCHAR(100),
    patient_id INT PRIMARY KEY,
    d_ID INT NOT NULL,
    c_ID INT NOT NULL,
    FOREIGN KEY (d_ID) REFERENCES Doctor(doctor_id),
    FOREIGN KEY (c_ID) REFERENCES Clinic(clinic_id));

CREATE TABLE DoctorSchedule (
    day_of_week VARCHAR(15) NOT NULL,
    start_time TIME NOT NULL,
    end_time TIME NOT NULL,
    schedule_id INT PRIMARY KEY,
    d_ID INT NOT NULL,
    c_ID INT NOT NULL,
    FOREIGN KEY (d_ID) REFERENCES Doctor(doctor_id),
    FOREIGN KEY (c_ID) REFERENCES Clinic(clinic_id));
CREATE TABLE Appointment(
    start_time DATETIME NOT NULL,
    end_time DATETIME,
```

```
    status VARCHAR(20),
    reason VARCHAR(200),
    appointment_id INT PRIMARY KEY,
    p_ID INT NOT NULL,
    d_ID INT NOT NULL,
    c_ID INT NOT NULL,
    FOREIGN KEY (p_ID) REFERENCES Patient(patient_id),
    FOREIGN KEY (d_ID) REFERENCES Doctor(doctor_id),
    FOREIGN KEY (c_ID) REFERENCES Clinic(clinic_id));

CREATE TABLE Prescription(
    issued_at DATETIME NOT NULL,
    notes VARCHAR(500),
    prescription_id INT PRIMARY KEY,
    p_ID INT NOT NULL,
    d_ID INT NOT NULL,
    appt_ID INT NOT NULL,
    FOREIGN KEY (p_ID) REFERENCES Patient(patient_id),
    FOREIGN KEY (d_ID) REFERENCES Doctor(doctor_id),
    FOREIGN KEY (appt_ID) REFERENCES Appointment(appointment_id));

    CREATE TABLE Medicine(
    name VARCHAR(200) NOT NULL,
    form VARCHAR(50) NOT NULL,
    strength VARCHAR(50) NOT NULL,
    is_controlled BIT, -- In SQL server the data type 'BOOLEAN' is invalid, 'BIT' is replaced (0=FALSE, 1=TRUE)
    medicine_id INT PRIMARY KEY);
CREATE TABLE PrescriptionItem(
    dosage VARCHAR(100),
    frequency VARCHAR(100),
    duration_days INT,
    instructions VARCHAR(300),
    prescr_ID INT NOT NULL,
```

```
    m_ID INT NOT NULL,
    FOREIGN KEY (prescr_ID) REFERENCES Prescription(prescription_id),
    FOREIGN KEY (m_ID) REFERENCES Medicine(medicine_id),
    PRIMARY KEY (prescr_ID, m_ID));

CREATE TABLE Inventory(
    quantity INT NOT NULL,
    reorder_level INT,
    last_updated DATETIME,
    inventory_id INT,
    c_ID INT NOT NULL,
    m_ID INT NOT NULL,
    PRIMARY KEY (inventory_id),
    FOREIGN KEY (c_ID) REFERENCES Clinic(clinic_id),
    FOREIGN KEY (m_ID) REFERENCES Medicine(medicine_id));

CREATE TABLE Invoice(
    total_amount DECIMAL(10,2) NOT NULL,
    paid_amount DECIMAL(10,2) NOT NULL,
    status VARCHAR(20),
    issued_at DATETIME NOT NULL,
    invoice_id INT PRIMARY KEY,
    c_ID INT NOT NULL,
    p_ID INT NOT NULL,
    appt_ID INT NOT NULL,
    FOREIGN KEY (c_ID) REFERENCES Clinic(clinic_id),
    FOREIGN KEY (p_ID) REFERENCES Patient(patient_id),
    FOREIGN KEY (appt_ID) REFERENCES Appointment(appointment_id));

CREATE TABLE InvoiceItem(
    description VARCHAR(300),
    amount DECIMAL(10,2) NOT NULL,
    invoice_item_id INT PRIMARY KEY,
```

```
    inv_ID INT NOT NULL,
    FOREIGN KEY (inv_ID) REFERENCES Invoice(invoice_id) );

CREATE TABLE Role(
    role_id INT PRIMARY KEY,
    role_name VARCHAR(50) NOT NULL);

CREATE TABLE Users( -- ‘User’ is a reserved keyword in SQL server, so it was renamed to ‘Users’ to avoid syntax errors.
    username VARCHAR(90) UNIQUE NOT NULL,
    password_hash VARCHAR(200) NOT NULL,
    person_type VARCHAR(10) NOT NULL CHECK (person_type IN ('PATIENT' , 'DOCTOR' , 'STAFF')),
    person_id INT NOT NULL,
    user_id INT PRIMARY KEY,
    role_ID INT NOT NULL,
    FOREIGN KEY (role_ID) REFERENCES Role(role_id));

CREATE TABLE Permission(
    code VARCHAR(50) UNIQUE NOT NULL,
    description VARCHAR(300),
    permission_id INT PRIMARY KEY);

CREATE TABLE RolePermission(
    role_ID INT NOT NULL,
    perm_ID INT NOT NULL,
    PRIMARY KEY (role_ID, perm_ID),
    FOREIGN KEY (role_ID) REFERENCES Role(role_id),
    FOREIGN KEY (perm_ID) REFERENCES Permission(permission_id));
```

- **Data Population:**

  The database tables were populated with representative sample healthcare data before executing the following SQL queries. The complete data insertion scripts are available in the project source code.

- **Sample Queries and Results:**

1- Listing the names and contact details of patients with more than two appointments in the last month:

```
SELECT
    p.full_name,
    p.phone,
    p.email,
    COUNT(a.appointment_id) AS appointment_count
FROM Patient p
JOIN Appointment a
    ON p.patient_id = a.p_ID
WHERE a.start_time >= DATEADD(MONTH, -1, GETDATE())
GROUP BY p.full_name, p.phone, p.email
HAVING COUNT(a.appointment_id) > 2;
```

- **Result**:

	full_name	phone	email	appointment_count
1	Mariam Al-Musa	0556632980	mariam.almusa@example.com	3
2	Nora Alkhaled	0552549445	nora.alkhaled@example.com	3

2- List medicines running low in stock at any location:

```
SELECT
    c.name AS clinic_name,
    c.city,
    m.name AS medicine_name,
    i.quantity,
    i.reorder_level
FROM Inventory i
JOIN Clinic c ON i.c_ID = c.clinic_id
JOIN Medicine m ON i.m_ID = m.medicine_id
WHERE i.quantity < i.reorder_level
```

ORDER BY c.name, m.name;

- **Result:**

	clinic_name	city	medicine_name	quantity	reorder_level
1	Al Amal Clinic	Jeddah	Dental Paste	20	50
2	Dar Al Seha Clinic	Riyadh	Ear Drops	15	30
3	Najd Health Center	Jeddah	Cream I	30	50
4	Salam Clinic	Riyadh	Vitamin D	20	50

3- Display appointments longer than 30 minutes, with patient and doctor details:

```
SELECT
    a.appointment_id,
    a.start_time,
    a.end_time,
    DATEDIFF(MINUTE, a.start_time, a.end_time) AS appointment_duration,
    p.full_name AS patient_name,
    d.full_name AS doctor_name,
    d.specialty,
    c.name AS clinic_name
FROM Appointment a
JOIN Patient p ON a.p_ID = p.patient_id
JOIN Doctor d ON a.d_ID = d.doctor_id
JOIN Clinic c ON a.c_ID = c.clinic_id
WHERE DATEDIFF(MINUTE, a.start_time, a.end_time) > 30
ORDER BY appointment_duration;
```

- **Result:**

	appointment_id	start_time	end_time	appointment_duration	patient_name	doctor_name	specialty	clinic_name
1	505	2025-11-21 13:00:00.000	2025-11-21 13:35:00.000	35	Huda Al-Qahtani	Dr. Mohammed Al-Rashid	Orthopedics	Al Shifa Center
2	514	2025-11-24 09:00:00.000	2025-11-24 09:35:00.000	35	Nora Alkhaled	Dr. Khalid Al-Fahad	Neurology	Najd Health Center
3	506	2025-11-22 14:00:00.000	2025-11-22 14:40:00.000	40	Youssef Ahmad	Dr. Aisha Al-Zahrani	Gynecology	Al Amal Clinic
4	501	2025-11-17 09:00:00.000	2025-11-17 09:40:00.000	40	Mariam Al-Musa	Dr. Ahmed Al-Tamimi	Cardiology	Al Noor Clinic
5	504	2025-11-20 12:00:00.000	2025-11-20 13:00:00.000	60	Omar Al-Fahad	Dr. Laila Alsaleh	Internal Medici…	Dar Al Seha Clinic
6	509	2025-11-25 09:30:00.000	2025-11-25 10:30:00.000	60	Dana Al-Rashid	Dr. Nasser Al-Qahtani	Internal Medici…	Hayat Clinic

4- Produce a report of revenue generated per clinic by city:

```
SELECT

    cl.city,

    cl.name AS clinic_name,

    SUM(inv.total_amount) AS total_revenue

FROM Clinic cl

JOIN Invoice inv ON cl.clinic_id = inv.c_ID

GROUP BY cl.city, cl.name

ORDER BY cl.city, total_revenue;
```

- **Result:**

	city	clinic_name	total_revenue
1	Dammam	Al Ishraq Clinic	180.00
2	Dammam	Al Naseem Center	190.00
3	Jeddah	Najd Health Center	170.00
4	Jeddah	Hayat Clinic	200.00
5	Jeddah	Al Amal Clinic	250.00
6	Madinah	Al Shifa Center	250.00
7	Makkah	Al Noor Clinic	175.00
8	Riyadh	Salam Clinic	140.00
9	Riyadh	Wellness Center	160.00
10	Riyadh	Dar Al Seha Clinic	300.00

Documentation & Security:

1- Authentication

- Users accounts are stored in a Users table.
- Store passwords as secure hashes (Bcrypt/Argon2) to prevent exposure of real passwords.
- MFA for staff; account lockout and session timeout.

2- Authorization (Role-Based Access Control)

- Each user in the system is assigned a specific role that determines the actions they are allowed to take.
- The roles used in the system are: Admin, Doctor, Nurse, Receptionist, Pharmacist, Lab Technician, Accountant, Manager, IT support, Patient.

3- Data Encryption

Encryption is a process of converting data into a form called chipertext to hide its meaning from unauthorized persons. The system uses encryption when the data is transferred and while the data is stored in the database.

- Data transmission between the application and database is secured using SSL/TLS protocols. To prevent the data from being intercepted.
- Sensitive data, like prescription details and billing records, are kept in the database in an encrypted format. To ensure that even if unauthorized access does occur at the storage level, all the information remains unreadable.
- In the system, passwords are stored using hashing algorithms (Bcrypt/Argon2), to ensure that the original passwords cannot be recovered even if the database is compromised.

4- Privacy & Sensitive Data Protection

- Sensitive data such as prescriptions and billing records can only be viewed by authorized roles.
- Patients are only allowed to view their own records.
- Staff can only access data needed for their job (least-privilage principal).

5- Role-permission table

Role	Allowed actions
Admin	Full access to all data and full user management
Doctor	Access to own patients, appointments, and inventory items related to prescription
Nurse	Manage appointments and users when necessary
Receptionist	Manage appointments and basic patient contact
Pharmacist	View inventory levels and managing inventory records
Lab Technician	View patient information
Accountant	View billing information, edit billing and payment details
Manager	View the clinic appointments
IT support	View appointments, patient information when required for support, and view billing information related to system checks